PRAGYA: AI Checkout Recovery Agent

Turning hesitation into conversion with real-time AI intervention

Built for

Kasparro

Agentic Commerce Hackathon

Track 2: AI Assisted Checkout Recovery

Technical Documentation

Team Name: Team Pragya

Members: Chandana S

Hitendra S

Date: April 2026

Repository:
https://github.com/chandana022005/AI-Checkout-Recovery-Agent

AI-powered Shopify-like checkout recovery system using real-time behavioural signals and contextual product recommendations.

Pragya – AI Checkout Recovery Assistant

Technical Document

---

1. System Overview

Pragya is a proactive, AI-driven checkout recovery agent designed to reduce cart abandonment through real-time, context-aware interventions. Unlike traditional reactive chatbots, Pragya utilizes a "Diagnostic Thinking Engine" powered by Google's Gemini API to analyse cart details, browsing history, and user interactions before deciding *whether* and *how* to intervene.

Core Objectives

1. **Reduce Abandonment**: Identify user friction during checkout (e.g., shipping costs, price hesitation) and provide immediate resolution.

2. **Increase Average Order Value (AOV)**: Intelligently suggest relevant add-on products to help users qualify for free shipping.

3. **Minimize Intrusion**: Remain invisible unless intervention is deemed necessary or the user explicitly asks a question.

---

2. System Architecture

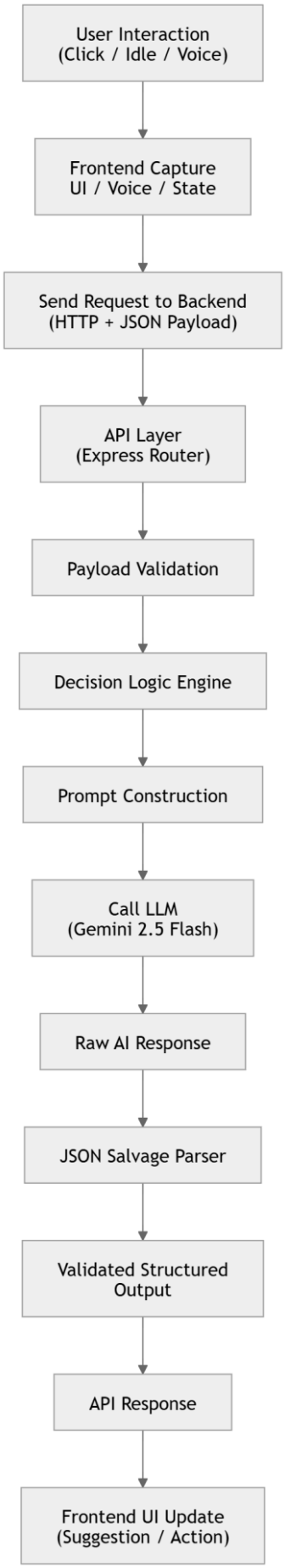
The project follows a standard client-server architecture:

The flow starts when a user interacts with the UI (click, idle event, or voice input). The frontend collects this input along with cart and session state, then sends it as a JSON request to the backend API.

The backend validates the request and passes it to the decision logic engine, which determines what to ask the AI and constructs a prompt. This prompt is sent to **Gemini 2.5 Flash**.

The AI returns a raw response, which is cleaned and structured by the JSON parser. The backend then sends a final, validated response back to the frontend.

Finally, the UI updates by showing a suggestion, message, or action to the user.


User Interaction
(Click / Idle / Voice)
Frontend Capture
UI / Voice / State
Send Request to Backend
(HTTP + JSON Payload)
API Layer
(Express Router)
Payload Validation
Decision Logic Engine
Prompt Construction
Call LLM
(Gemini 2.5 Flash)
Raw AI Response
JSON Salvage Parser
Validated Structured
Output
API Response
Frontend UI Update
(Suggestion / Action)

3. Frontend Architecture

The frontend is built for high-fidelity performance and a premium e-commerce look.

Technologies Used

Framework: React 18 with TypeScript

Build Tool: Vite

Styling: Tailwind CSS with `tailwindcss-animate`

Components: `shadcn/ui` (built on top of Radix UI primitives for accessibility)

Icons: Lucide React

Key Features

-Glassmorphism UI: The widget design closely mirrors native checkout forms to avoid a "third-party" feel.

Dynamic Context: The frontend actively tracks the cart total, selected items, shipping costs, and idle time.

Voice Input Integration: Implements the native Web Speech API, allowing users to interact with the AI assistant hands-free.

1-Click Add-ons: Suggested products returned by the AI can be instantly added to the cart directly from the chat widget, triggering state recalculations.

---

4. Backend Architecture

The backend serves as the orchestration layer and prompt generation engine.

Technologies Used

Runtime: Node.js

Framework: Express.js

AI SDK: `@google/generative-ai`

Core Components

Decision Matrix & Prompting

The backend constructs a robust prompt combining:

Store Policy Context: Static delivery and return rules.

Dynamic User Context: Cart delta (amount needed for free shipping), current items, search history, and chat history.

Special Behaviours: Strict instructions on when to push products vs. explain value vs. clarify policies.

JSON Enforcement & Salvage Parser

The backend demands a strict JSON schema from the Gemini model. Because LLMs occasionally hit `MAX_TOKENS` limits and truncate their JSON output, the backend features a robust **Salvage Parser**:

- Attempts direct `JSON.parse() `.

- If it fails, applies regex and string manipulation to automatically close open arrays/objects and inject trailing braces.

- Provides a graceful fallback object if salvage is impossible, ensuring the frontend never crashes.

---

5. API Specification

Endpoint: `/api/recover`

Method: `POST`

Description: Analyzes the current checkout state and returns the optimal AI intervention.

Request Payload:

```
json
{
  "cart_total": 1250,
  "items": [{ "name": "Classic White Tee", "price": 1250 }],
  "shipping_cost": 150,
  "shipping_type": "standard",
  "user_message": "Is delivery free?",
  "chat_history": [],
  "event": "user_enquiry",
  "idle_time": 5,
  "user_typing": false,
  "search_history": ["men's clothing"]
}
```

Response Payload:

json

```
{
  "confidence": 0.95,
  "reasoning": "User asked about delivery; they are ₹250 away from free shipping. Suggesting an add-on.",
  "intent": "clarify_info",
  "strategy": "suggest_addon",
  "response": "Standard delivery is ₹150, but you're only ₹250 away from free shipping! Would you like to add some Premium Cotton Socks to your order?",
  "suggested_action": {
    "type": "add_product",
    "reason": "free_shipping",
    "product": {
      "name": "Premium Cotton Socks",
      "price": 199
    }
  },
  "intervention_triggered": true
}
```

---

6. Environment Setup

Backend `.env`

To run the backend, the following environment variables are required:

Gemini_api_key - The API key from Google AI Studio to access Gemini 2.5 Flash.

Port -The port the Express server runs on (defaults to 3001).

---

7. Strategic Logic Matrix

Pragya evaluates user intent and applies one of the following strategies:

Strategy	Trigger Condition	AI Action
suggest_addon	User mentions shipping/price **or** is idle, and cart gap ≤ ₹300	Recommends a relevant product to help unlock free shipping
explain_value	User shows hesitation about the total cost	Highlights product quality, benefits, and craftsmanship to justify the price
clarify_info	User asks about delivery time or shipping cost	Explains standard vs express delivery, including timelines and charges
reassure_user	User asks about returns, safety, or trust-related concerns	Communicates return policy and secure checkout assurances
no_intervention	User is typing, engaged, or explicitly declines help	Takes no action to avoid interrupting the user experience